Exercise sheet 07 - Computational Statistical Physics

Iason Kazazis | Physics MSc / 25-935-701 | 27/4/2026

Exercise 1. Event-driven molecular dynamics

The purpose of this exercise is to simulate particles interacting through a hard-core potential. Unlike molecular dynamics with a continuous potential, event-driven molecular dynamics does not integrate forces at every small time step. Between collisions the motion is ballistic, and the algorithm advances the system directly to the next collision event.

I use units with particle mass $m = 1$. The mandatory simulations are one-dimensional hard rods with radius $R = 0$, as suggested in the exercise sheet. The coefficient of restitution e is used for particle-particle and particle-wall collisions.

Task 1. One-dimensional event-driven implementation

Collision search

The particles are stored in increasing order, $x_0 < x_1 < \ldots < x_{N-1}$. In one dimension hard rods cannot overtake each other, therefore only neighbouring rods need to be checked. For neighbouring particles i and $i+1$, the candidate pair-collision time is

$$t_{i,i+1} = (x_{i+1} - x_i - 2R)/(v_i - v_{i+1}), \quad \text{if} \quad v_i - v_{i+1} > \varepsilon,$$

and it is set to infinity otherwise. Here ε is a small threshold that prevents round-off noise from being interpreted as a collision. The wall candidate times are

$$t^{\text{left}}_i = (R - x_i)/v_i \text{ for } v_i < 0, \quad t^{\text{right}}_i = (L - R - x_i)/v_i \text{ for } v_i > 0.$$

The next global event is the smallest positive candidate among the neighbouring-pair and wall times.

> **Implementation convention.** The event type is returned explicitly in both branches of `collision_finder`. In particular, pair collisions return `wall_collision=False`. In the rare case of an exact numerical tie between a wall and pair candidate, the code resolves the wall event first. Random initial conditions make such exact ties measure-zero events, and this convention does not affect the simulations shown here.

Position and velocity updates

Between collisions there are no forces, so the position update is exact:

$$x_i(t + t_c) = x_i(t) + v_i(t)\, t_c.$$

For two equal-mass particles i and $j=i+1$, momentum conservation and the restitution rule for the normal relative velocity give

$$v'_i = [(1-e)/2]\, v_i + [(1+e)/2]\, v_j, \quad v'_j = [(1+e)/2]\, v_i + [(1-e)/2]\, v_j.$$

For a wall collision the update is $v'_i = -e\, v_i$. After each event the contact position is corrected explicitly, which suppresses the accumulation of tiny overlaps.

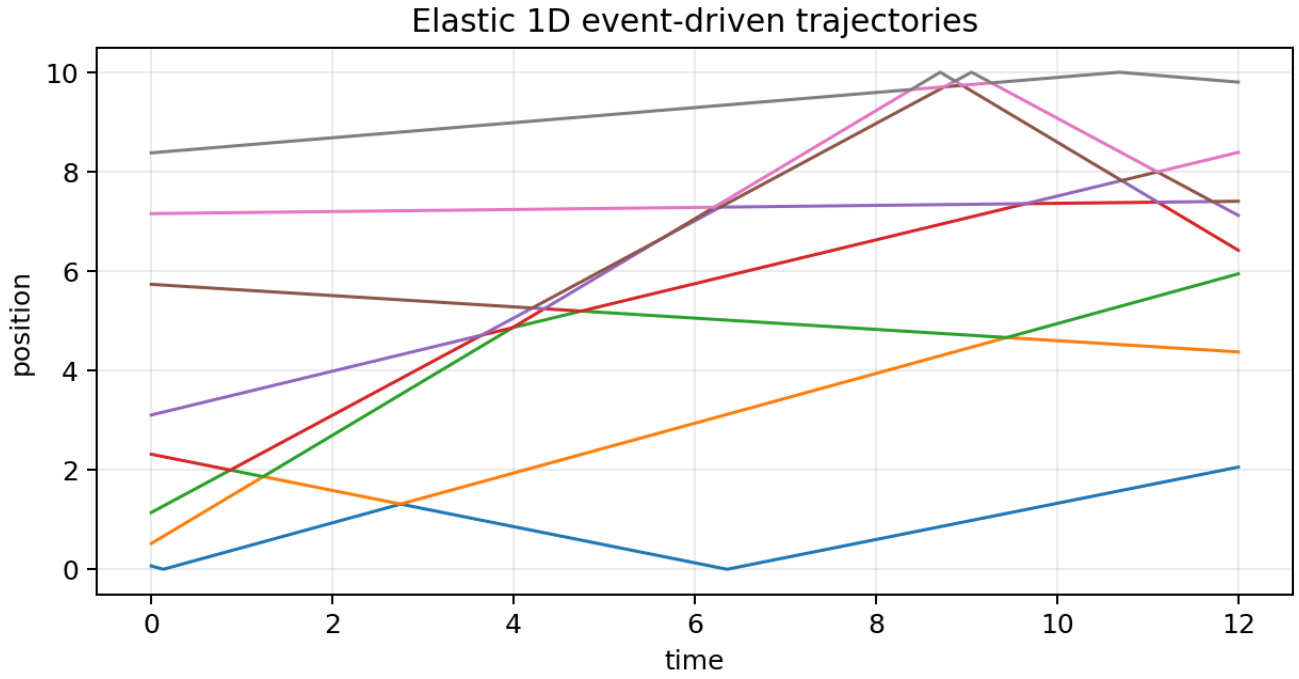

Figure 1: Example of the event-driven trajectories for eight elastic hard rods. The trajectories are straight between events and change slope only at collisions.

As a basic check, two equal-mass particles with velocities (1.0, −0.5) leave their first elastic collision with velocities (−0.5, 1.0). A random elastic box run with $N = 20$ conserved the kinetic energy to the displayed precision over 1000 events.

Task 2(a). Random particles in a one-dimensional box

I simulated $N = 60$ rods in a box of length $L = 10$. The initial positions were sampled uniformly at random and then sorted, and the velocities were sampled uniformly from $[-1, 1]$. The same initial state was used for the elastic case $e = 1$ and the inelastic case $e = 0.8$. The run was limited to 10^4 events and to a maximum physical time $t_{\max} = 50$.

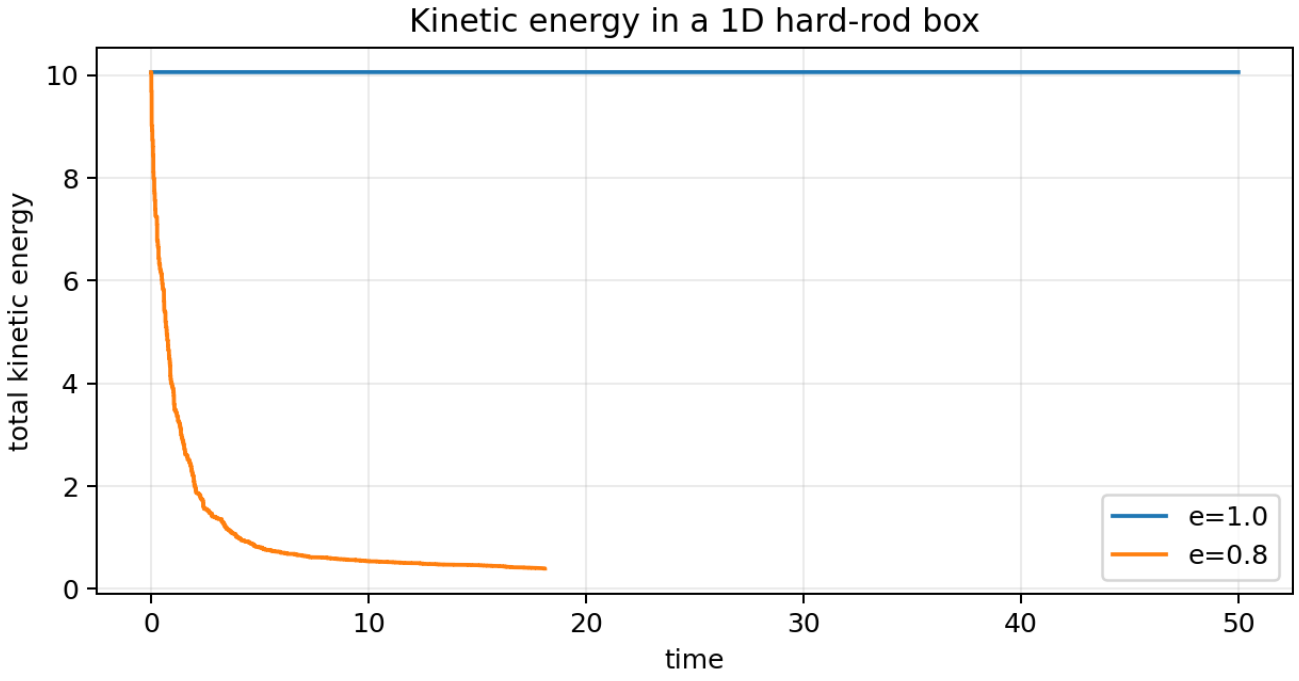

Figure 2: Total kinetic energy in a one-dimensional hard-rod box. For $e = 1$ the total kinetic energy is conserved. For $e = 0.8$ it decreases at collision events.

e	recorded events	E_i	E_f	E_f/E_i
1.000000	6138	10.056538	10.056538	1.000000
0.800000	10000	10.056538	0.394597	0.039238

The result agrees with the collision rules. In the elastic case, pair collisions exchange velocities and wall collisions only reverse a sign, so the kinetic energy remains constant. In the inelastic case, each collision reduces the normal component of the relative velocity, and the total kinetic energy decreases in steps because it changes only at events.

Task 2(b). Cluster of particles hitting a wall

The cluster was initialised as evenly spaced particles with a common velocity directed towards the left wall. I used a semi-infinite interpretation of the setup: the left wall is physical, while the right wall is an artificial far-away boundary. In the code the right wall is guarded explicitly: if it is reached before rebound detection, the simulation raises an error and the margin must be increased.

The stopping criterion after rebound is

$$v_i \geq 0 \text{ for all i}, \quad \text{and} \quad v_i \leq v_{i+1} \text{ for all neighbouring pairs.}$$

At this point the particles move away from the left wall and no adjacent pair is closing. The effective coefficient of restitution is

$$e_{eff} = \text{sqrt}(E_f/E_i).$$

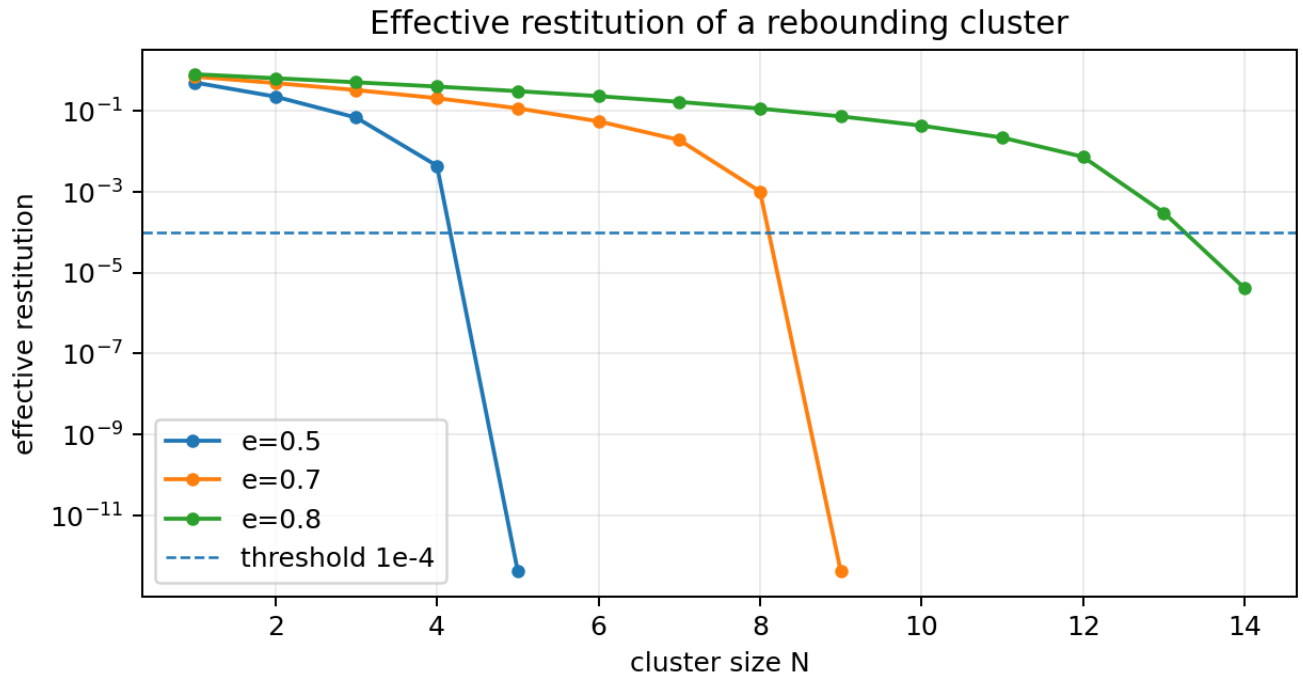

Figure 3: Effective restitution of a rebounding cluster as a function of the number of particles. The dashed line marks the threshold $e_{eff} = 10^{-4}$.

microscopic e	N_c for e_eff < 1e-4	last e_eff	events in last run
0.500000	5	4.2265e-13	446
0.700000	9	4.1440e-13	1742
0.800000	14	4.1132e-06	2601

The numerical estimate is $N_c = 5$ for $e = 0.5$, $N_c = 9$ for $e = 0.7$, and $N_c = 14$ for $e = 0.8$. As expected, the required cluster size increases when the microscopic collisions become more elastic. The macroscopic restitution of a cluster is therefore not simply the microscopic restitution coefficient; the incoming energy is redistributed through many internal collisions, each of which dissipates part of the relative kinetic energy.

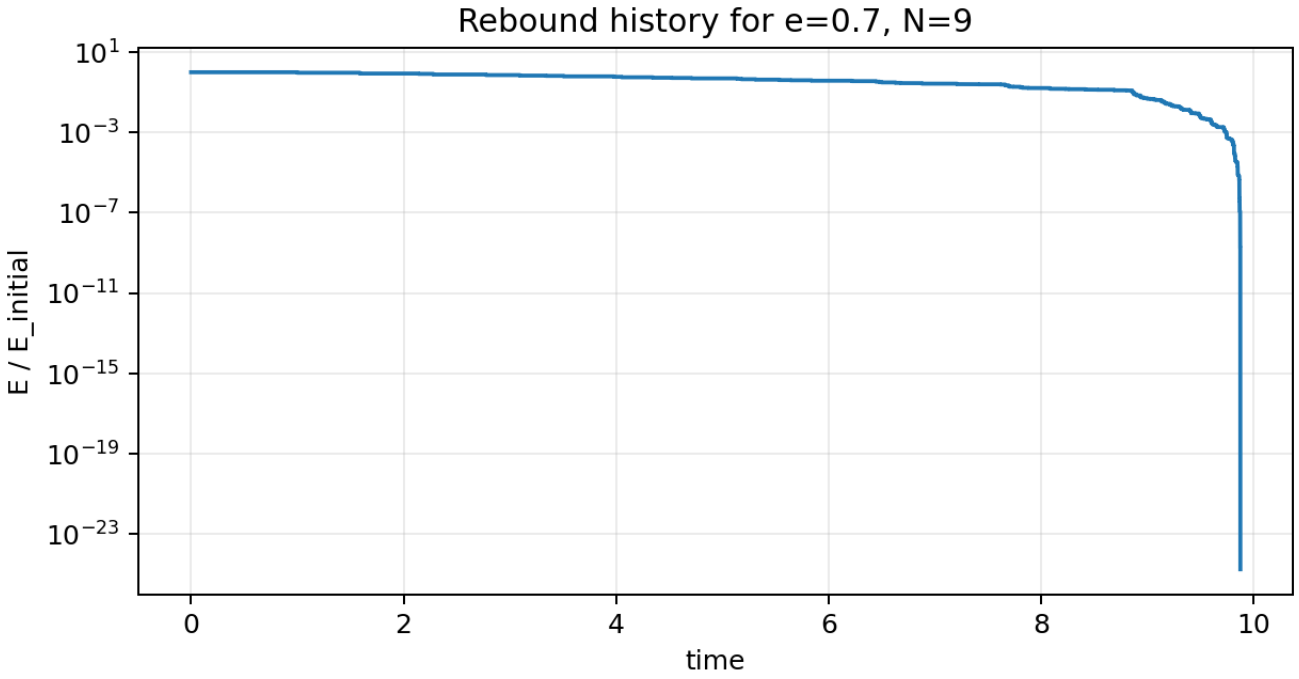

Figure 4: Example rebound history for $e = 0.7$ and $N = 9$. The energy is normalised by its initial value. It remains constant between events and decreases at collisions.

> **Numerical clean-up.** After very dissipative collision cascades, velocities below 10^{-30} are rounded to zero. This is far below the velocity scale used in the simulations and only removes floating-point ghosts which could otherwise generate artificial late-time events.

Optional Task 3. Two-dimensional hard disks

The optional two-dimensional extension uses a direct $O(N^2)$ search. For a pair of disks the collision time is obtained from

$$|r_{ij} + v_{ij}\, t|^2 = (2R)^2,$$

which gives a quadratic equation in t. The event is accepted only if the particles are approaching, the discriminant is non-negative, and the smallest root is positive. For smooth hard disks, only the velocity component along the line of centres is updated, while the tangential component is unchanged.

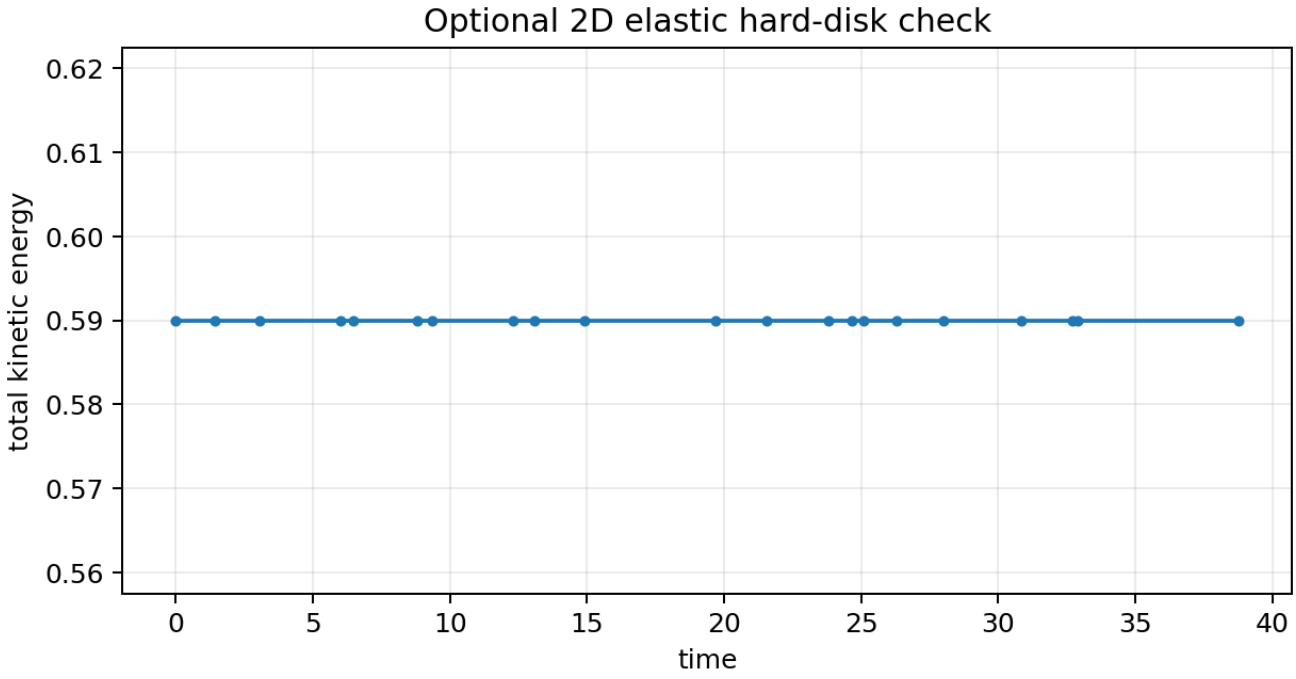

Figure 5: Small elastic two-dimensional check. For $e = 1$ the kinetic energy is conserved to numerical precision.

Optional Task 4. Priority queue

The simple one-dimensional implementation recomputes all neighbouring-pair and wall event candidates after each collision. Since the one-dimensional interaction is only between neighbours, this is already O(N) per event. A more efficient event-driven implementation stores predicted events in a heap. After a collision, only the involved particles and neighbouring pairs are rescheduled.

Previously predicted events can become invalid because one of their particles has collided in the meantime. I used lazy invalidation: each particle has a collision counter, and each heap event stores the counter values at insertion. A popped event is accepted only if the stored counters still match the current counters. In an elastic comparison over 200 events, the heap implementation reproduced the same event times up to 2.842×10^{-14} and the same kinetic energies to the displayed precision.

Conclusion

The one-dimensional event-driven algorithm was implemented following the skeleton structure. In the elastic case the kinetic energy is conserved to numerical precision. In the inelastic case, energy decreases only at collision events, as expected from the restitution rule. For a cluster hitting a wall, the effective restitution rapidly decreases with cluster size; for the tested values I found $N_c = 5$, 9 and 14 for $e = 0.5$, 0.7 and 0.8 respectively. The numerical conventions used in the implementation are explicit pair-event return values, deterministic tie-breaking, a guarded artificial right wall in the cluster setup, and a near-zero velocity clean-up after strongly dissipative cascades.

References

- Computational Statistical Physics, Exercise sheet 7: Event-driven molecular dynamics.
- Provided Python skeleton `edmd_1d_skeleton.py`.
- Course lecture material on hard-sphere potentials, event-driven dynamics and event queues.